

Impacto de las Épocas Offline en el Rendimiento y la Exactitud de Entrenamientos Distribuidos con O.N.O.

Marcos Bianchi

Facultad de Ingeniería, Universidad de Buenos Aires

Aprendizaje Profundo

Julio 2026

Resumen—Retrasar la sincronización entre nodos en un entrenamiento distribuido reduce la frecuencia de comunicación pero induce divergencia entre los pesos (*client drift*). Este trabajo evalúa el impacto de variar las épocas offline (E) utilizando el sistema de entrenamiento O.N.O. bajo las topologías *Parameter Server* sincrónico y *All-Reduce (ring)*. Los experimentos se ejecutaron sobre un clúster de contenedores Docker con emulación de red mediante Pumba. Entrenando LeNet-5 sobre MNIST FULL, los resultados muestran que incrementar E no aporta reducciones significativas en los tiempos totales debido a la contención del hardware local, mientras que degrada la *accuracy* final e incrementa la inestabilidad de la pérdida durante el entrenamiento.

Palabras clave—Épocas Offline, *Parameter Server*, *All-Reduce*, O.N.O., *Client Drift*, Pumba.

I. INTRODUCCIÓN Y OBJETIVOS

Permitir que los nodos operen de forma autónoma durante algunas épocas posterga el intercambio de parámetros, disminuyendo la dependencia de la red y priorizando el cómputo local. Sin embargo, demorar la sincronización introduce divergencia en los pesos de los nodos, un fenómeno conocido como *client drift* [1].

Pregunta de investigación. ¿Cómo impacta la variación de las épocas offline (E) en la velocidad de convergencia, el tiempo de comunicación y la eficacia del modelo entrenado bajo distintas topologías de red en un entorno distribuido con restricciones de red reales?

Objetivos. Evaluar esta pregunta utilizando O.N.O., un sistema de entrenamiento distribuido de modelos de aprendizaje profundo desarrollado para este trabajo. Específicamente, se busca:

- Comparar el desempeño de los esquemas de sincronización *Parameter Server* sincrónico y *Ring All-Reduce* al variar las épocas offline E .
- Evaluar el comportamiento del sistema en un entorno de red realista.
- Cuantificar el *trade-off* entre la reducción del tráfico de red y la degradación de la *accuracy* final debido al *client drift*.

II. ARQUITECTURA DEL SISTEMA Y MECANISMOS DE SINCRONIZACIÓN

II-A. Componentes Principales de O.N.O.

- **Orchestrator:** Mediante una configuración declarativa del entrenamiento brindada por el usuario, configura al resto de nodos para iniciar el entrenamiento de un modelo usando alguno de los algoritmos implementados. Al finalizar un entrenamiento, consolida los resultados parciales generando un archivo *safetensors* que contiene los parámetros del modelo entrenado. Para una mejor automatización, cuenta con una interfaz FFI de Python que fue utilizada para orquestar los ensayos discutidos más adelante.
- **Workers:** Computan gradientes locales por *backpropagation* sobre una partición asignada del dataset. En *All-Reduce* aplican localmente el optimizador, mientras que en *Parameter Server* delegan la actualización a los servidores.
- **Servers:** Nodos dedicados al almacenamiento y optimización de parámetros del modelo. Reciben gradientes parciales de los *workers*, los agregan y aplican sobre los parámetros para luego redistribuirlos.

II-B. Dinámica de Sincronización Demorada ($E > 1$)

Al parametrizar $E > 1$, los nodos ejecutan E épocas de entrenamiento de forma autónoma antes de realizar un intercambio de red. El tráfico de comunicación se reduce considerablemente de forma inversamente proporcional a E . No obstante, la falta de sincronización continua causa que los pesos locales de los *workers* diverjan (*client drift*). Al alcanzar la fase de sincronización, la consolidación se realiza agregando los gradientes acumulados durante dicho período.

Bajo un entrenamiento optimizado por *Gradient Descent*, la actualización del vector de parámetros global tras un intervalo de sincronización se modela como:

$$W_{t+E} = W_t - \frac{\lambda}{N} \sum_{i=1}^N G_i^{(E)} \quad (1)$$

donde N es el número de *workers*, λ es el *learning rate* y $G_i^{(E)}$ representa la acumulación de gradientes calculados por el *worker* i a lo largo de las E épocas locales.

II-C. Algoritmos de Entrenamiento Distribuido

El sistema implementa distintos algoritmos de comunicación distribuida, centrándose este trabajo en:

- **Parameter Server Sincrónico:** Algoritmo centralizado basado en el modelo propuesto por Li et al. [2]. Una barrera de sincronización bloquea a los *workers* hasta que los *servers* procesan el lote global de gradientes, garantizando que todos los nodos inicien el siguiente ciclo desde el mismo estado de parámetros.
- **All-Reduce (Ring):** Algoritmo descentralizado inspirado en arquitecturas de alto rendimiento como Horovod [3]. Los *workers* se organizan en una topología de anillo lógico y comparten fragmentos de gradientes en fases sucesivas de *scatter* y *gather*, actualizando de forma idéntica y local el vector de parámetros resultante.

III. METODOLOGÍA EXPERIMENTAL

Todos los entrenamientos fueron realizados de forma secuencial utilizando Docker para instanciar un clúster multicompuntadora sobre el cual Pumba emuló las condiciones de una red hogareña típica. El tiempo total de ejecución acumulado fue de 11 horas y 15 minutos. Las variables del modelo y del sistema O.N.O se consolidan en la Tabla I, mientras que las especificaciones técnicas del hardware y el entorno de red emulado se detallan en la Tabla II.

Tabla I: Configuración de Ensayos

Categoría	Parámetro	Valores Evaluados
Modelo y Datos	Dataset	MNIST (Full) [4]
	Arquitectura de Red	LeNet-5
	Función de Pérdida	Entropía Cruzada
Optimización	Optimizador	Adam
		$\alpha = 0,01$
		$\beta_1 = 0,9$
		$\beta_2 = 0,999$
		$\epsilon = 10^{-8}$
	Tamaño de Mini-Batch	64
	Épocas Máximas	48
Infraestructura	Serialización	Sparse ($r = 0,5$)
Variables Libres	Algoritmo Distribuido	All-Reduce, Parameter Server (Sincrónico)
	Nodos	2, 4, 6, 8, 10
	Épocas Offline	0, 1, 2, 3, 4

Para el algoritmo de *Parameter Server*, el total de nodos se compone de forma equitativa entre *workers* y *servers*.

III-A. Compresión de Red: Serialización Sparse

Para optimizar el volumen de comunicación en red, el sistema utiliza un esquema de serialización de gradientes *sparse* (o dispersa) [5], el cual limita el intercambio a los componentes de mayor relevancia. El parámetro $r \in [0, 1)$ determina la proporción de compresión: a mayor r , menor es el tamaño del mensaje transmitido.

La cardinalidad efectiva k del mensaje se calcula mediante:

$$k = \text{round}(|g| \cdot (1,0 - r)) \quad (2)$$

El valor k selecciona los k elementos de mayor magnitud absoluta dentro del tensor de gradientes local. Todo valor que alcance o supere dicho umbral se incorpora al mensaje. En estos experimentos se fijó $r = 0,5$, filtrando el 50 % de los componentes para reducir el tránsito en red. Los elementos no transmitidos se acumulan en un gradiente residual local para ser considerados en la siguiente actualización.

III-B. Emulación de Red con Pumba

Para evaluar el desempeño en un escenario distribuido realista sobre redes residenciales, se empleó Pumba [6], una herramienta de emulación de condiciones de red y pruebas de resiliencia. Mediante la integración con *docker-compose*, se inyectaron restricciones de ancho de banda, retardo (latencia con distribución Pareto) y pérdida correlacionada de paquetes sobre la interfaz de red de los contenedores, replicando el comportamiento de una conexión hogareña típica.

Tabla II: Entorno de Ejecución y Emulación de Red

Categoría	Componente	Detalle
Procesador	Modelo	Intel Core i5-8350U
	Arquitectura	x86_64
	Núcleos Físicos	4
	Hilos Lógicos	8
	Frecuencia de Reloj	Base: 1,7 GHz Turbo: 3,6 GHz
Memoria	Capacidad Total RAM	8 GB
	Tipo	DDR4
	Velocidad	2400 MHz
Red Emulada	Retardo (Delay)	Tiempo: 6 ms Jitter: 15 ms
	Pérdida de Paquetes	Distribución: Pareto Probabilidad: 0,3 %
	Ancho de Banda	Correlación: 25 % Tasa: 150 Mbit/s
Entorno de Software	Sistema Operativo	Ubuntu 24.04 LTS
	Entorno de Ejecución	Docker v29.6.1
		Rust v1.96.1
		CPython v3.14.0 Pumba 1.1.7

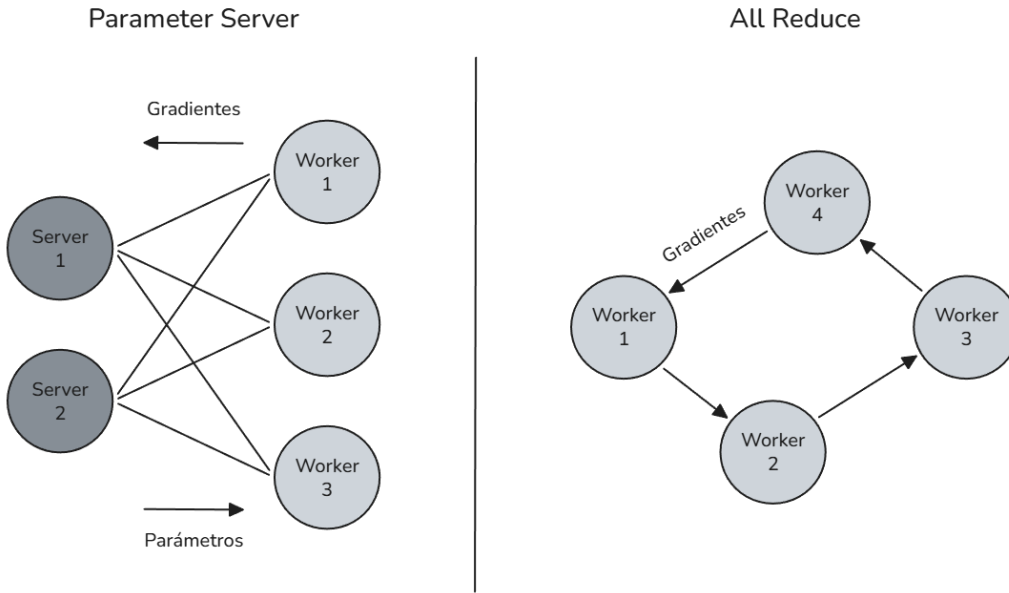


Figura 1: Topologías distribuidas soportadas en O.N.O. (Izquierda) Arquitectura centralizada bajo *Parameter Server* donde los *workers* obtienen parámetros y envían gradientes a *servers* fragmentados. (Derecha) Topología descentralizada en anillo *All-Reduce* donde los nodos se comunican de forma punto a punto con sus vecinos adyacentes.

IV. RESULTADOS Y ANÁLISIS EXPERIMENTAL

IV-A. Análisis Temporal y Escalabilidad del Hardware

La variación de E no indujo mejoras significativas en los tiempos de ejecución de los entrenamientos; las curvas de cada configuración exhiben un solapamiento estricto en todos los ensayos, incluso emulando condiciones de red con Pumba. Dado que el sistema se ejecuta en una única máquina, la supresión del costo de comunicación no logra compensar la sobrecarga de cómputo local, contrariamente a las expectativas iniciales de optimización.

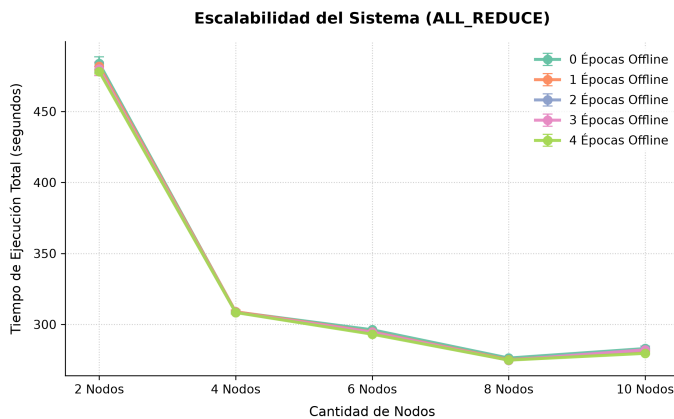


Figura 2: Comparación de tiempos de ejecución bajo distintas cantidades de Épocas Offline con All-Reduce.

Como se observa en la Figura 2, la escalabilidad del rendimiento se estanca al superar los 4 nodos. En *All-Reduce*,

la elevación a 10 nodos degrada el desempeño global debido a la alta contención por el CPU.

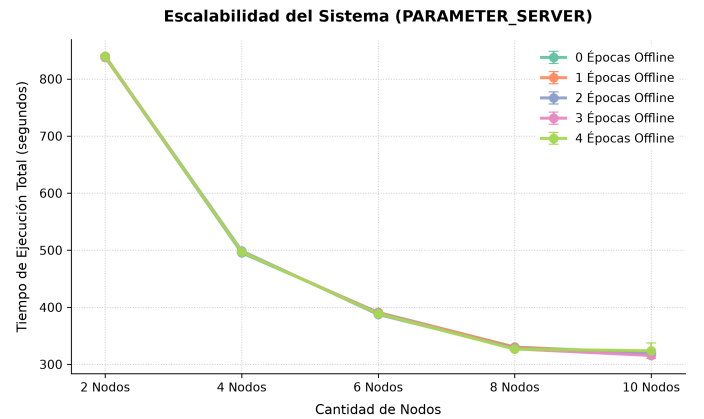


Figura 3: Comparación de tiempos de ejecución bajo distintas cantidades de Épocas Offline con *Parameter Server*.

En la Figura 3 se constata que la arquitectura de *Parameter Server* con 10 nodos (5 *workers* y 5 *servers*) reduce el tiempo respecto a configuraciones menores. A diferencia de *All-Reduce*, donde los procesos operan en cómputo continuo, el esquema sincrónico de *Parameter Server* introduce estados de inactividad obligatorios mediante barreras mientras el servidor actualiza el modelo. Este comportamiento disminuye la contención, permitiendo una alternancia eficiente de hilos de ejecución.

Cabe destacar que *All-Reduce* resulta intrínsecamente más veloz en configuraciones de pocos nodos. Esto responde a una distribución balanceada de responsabilidades donde cada

worker computa, transmite u optimiza parámetros de forma sostenida sin dependencias centralizadas.

IV-B. Métricas de Exactitud

El impacto de la variación de E se manifiesta con mayor claridad en las métricas de exactitud (*accuracy*), al ser variables independientes de las limitaciones del hardware de ejecución.

En ambos algoritmos, la exactitud final del modelo decrece de manera monótona a medida que se incrementa E . Este comportamiento es consistente con la teoría: la introducción de una única época offline ($E = 1$) reduce la frecuencia de sincronización global a la mitad, provocando un deterioro inmediato de aproximadamente el 1 % en la métrica final.

Vale remarcar que O.N.O. realiza las sincronizaciones a nivel de época completa (E). Si el esquema de sincronización operara a una granularidad más fina —como por paso de entrenamiento o *mini-batch*—, la acumulación de gradientes desacoplados durante múltiples iteraciones locales tendría una frecuencia de intercambio significativamente mayor, por lo que el impacto de variar las iteraciones *offline* resultaría notablemente más drástico tanto en el volumen de tráfico de red ahorrado como en la divergencia de los pesos.

En un entorno multi-computadora real, un valor mayor de E podría justificar esta pérdida si la aceleración en tiempo permitiera ejecutar un mayor número de épocas totales. Resta determinar si dicho incremento en volumen de entrenamiento compensaría la divergencia algorítmica.

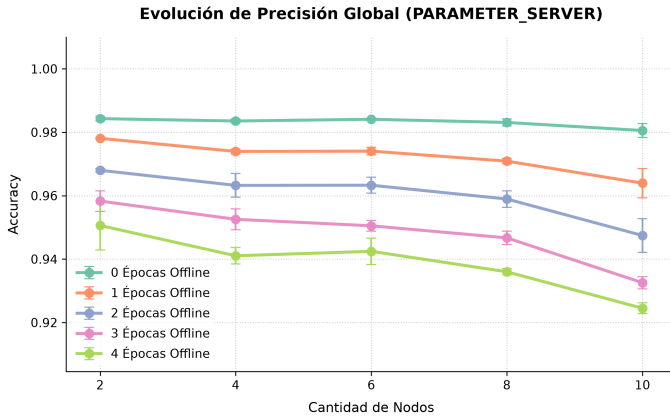


Figura 4: Comparación de accuracy bajo distintas cantidades de Épocas Offline con Parameter Server.

La degradación de la exactitud se acentúa de forma proporcional al número de nodos. Al incrementar las entidades distribuidas, las particiones locales del dataset se reducen y se vuelven potencialmente sesgadas. La falta de sincronización frecuente exacerba el impacto de estos sesgos locales en los *workers*, perjudicando la convergencia global del sistema.

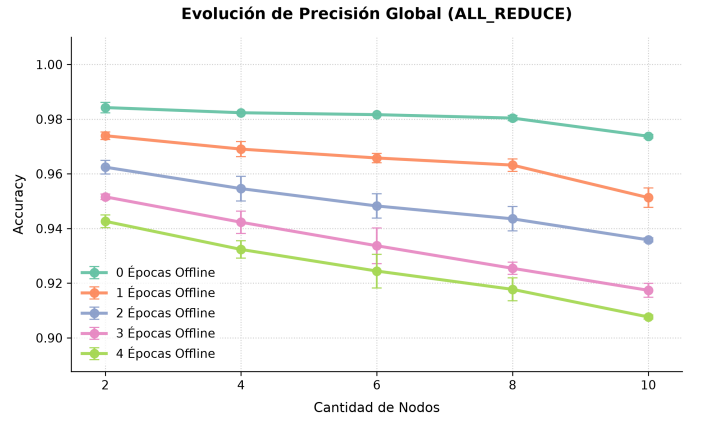


Figura 5: Comparación de accuracy bajo distintas cantidades de Épocas Offline con *All-Reduce*.

En *All-Reduce*, la pérdida de exactitud respecto a la cantidad de nodos y E es más severa que en *Parameter Server*. Bajo las condiciones experimentales, las pruebas de *All-Reduce* duplican el número de *workers* activos al prescindir de servidores dedicados. Esto fragmenta aún más el conjunto de datos e incrementa el aislamiento operativo. La integración tardía de parámetros fuerza correcciones abruptas en el gradiente global, desestabilizando el proceso de optimización.

IV-C. Inestabilidad de Pérdida durante el Entrenamiento

A partir del historial de entrenamiento se seleccionaron trayectorias representativas que confirman que el incremento de E introduce inestabilidad en la función de pérdida. En configuraciones de 2 nodos para ambos algoritmos, esta inestabilidad se evidencia mediante oscilaciones y picos de pérdida proporcionales al valor de E .

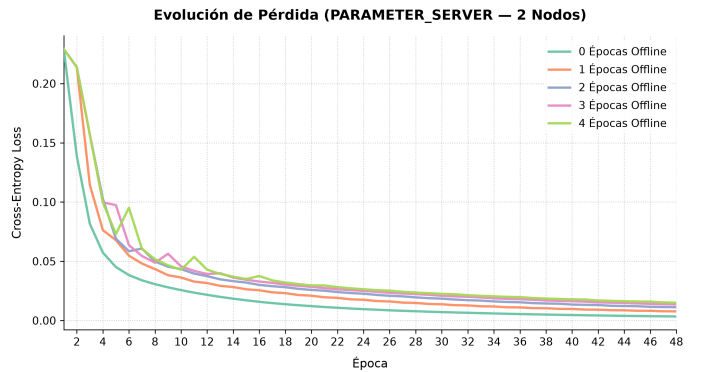


Figura 6: Trayectoria de Loss en Entropía Cruzada exhibiendo ruido estocástico por cómputo offline en Parameter Server (2 nodos).

La dinámica del optimizador presenta mayores dificultades en las épocas iniciales, fase asociada a las correcciones de mayor magnitud. Posteriormente, el sistema se estabiliza al aproximarse a un mínimo local. Sin embargo, los valores de convergencia residual son sistemáticamente más altos a mayor E . Este fenómeno sugiere que el desacoplamiento prolongado

restringe la capacidad del optimizador para alcanzar el mínimo global, induciendo un comportamiento análogo al de una tasa de aprendizaje (*learning rate*) excesivamente alta que provoca oscilaciones permanentes.

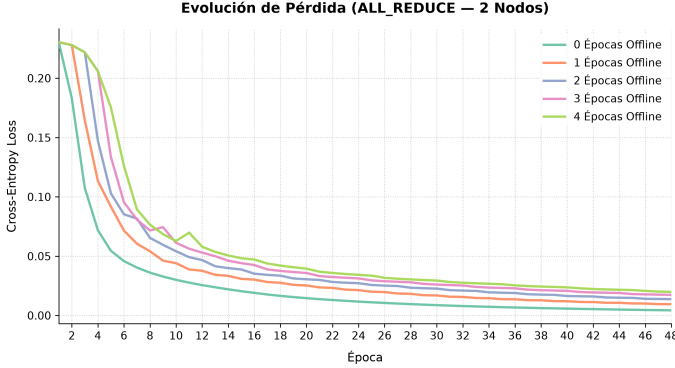


Figura 7: Trayectoria de Loss en Entropía Cruzada exhibiendo ruido estocástico por cómputo offline en *All-Reduce* (2 nodos).

En *All-Reduce* se observa un patrón similar, aunque con oscilaciones relativamente más atenuadas que en *Parameter Server*, revirtiendo la hipótesis inicial que preveía una mayor inestabilidad en el esquema descentralizado.

Considerando que la desestabilización afecta negativamente el resultado, resulta de interés evaluar estrategias de activación dinámica para las épocas offline. Una hipótesis plantea su introducción exclusiva en fases avanzadas del entrenamiento, cuando la optimización principal ha concluido, permitiendo que la exploración independiente de un *worker* en un mínimo local guíe favorablemente al resto de los nodos tras la sincronización.

V. DISCUSIÓN Y LIMITACIONES

La ventaja teórica de introducir épocas offline radica exclusivamente en reducir el tiempo total de entrenamiento al disminuir la frecuencia de comunicación. En el entorno local simulado, eliminar dicho costo no logra compensar la degradación en la exactitud del modelo, por lo que en este conjunto de experimentos no se evidenciaron beneficios operativos al utilizar valores de $E > 0$.

En contraposición, en un despliegue multicomputadora sobre una red física heterogénea, retrasar la agregación de gradientes y la sincronización de nodos puede ser crítico para amortizar la latencia y el ancho de banda consumido. En última instancia, la conveniencia de este enfoque dependerá de la relación entre el tamaño del modelo, las prestaciones de la red y la cantidad de nodos involucrados.

Resulta relevante analizar la naturaleza del modelo y del conjunto de datos. En arquitecturas con una alta cantidad de parámetros, los mensajes transmitidos (gradientes y pesos) demandan un ancho de banda considerable. En escenarios donde la comunicación domina sobre el tiempo de cómputo local (modelos grandes o redes de baja velocidad), incrementar E permite elevar la tasa de épocas por segundo. Por el contrario, en modelos pequeños o datasets masivos donde

el cuello de botella es la carga computacional local, postergar la sincronización solo introduce *client drift* sin aportar una aceleración significativa.

VI. CONCLUSIÓN

El uso de épocas offline en O.N.O. modifica el compromiso entre la frecuencia de comunicación y la convergencia algorítmica del entrenamiento. En la infraestructura evaluada, incrementar E degradó la exactitud del modelo y su estabilidad de manera sostenida, sin aportar reducciones en los tiempos totales debido a la contención del hardware local.

Como trabajo futuro, se plantea validar el comportamiento del sistema en un entorno multicomputadora físico real, evaluando modelos de mayor escala y analizando estrategias de sincronización adaptativas a lo largo del entrenamiento.

REFERENCIAS

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial Intelligence and Statistics*. PMLR, 2017, pp. 1273–1282.
- [2] M. Li, L. Zhou, Z. Yang, A. Li, F. Xia, D. G. Andersen, and A. Smola, "Parameter server for distributed machine learning," in *NIPS Workshop on Big Learning*, Lake Tahoe, NV, USA, 2013. [Online]. Available: <https://www.istc-cc.cmu.edu/publications/papers/2013/ps.pdf>
- [3] A. Sergeev and M. Del Balso, "Horovod: Fast and easy distributed deep learning in TensorFlow," *arXiv preprint arXiv:1802.05799*, 2018.
- [4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [5] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. Copenhagen, Denmark: Association for Computational Linguistics, September 2017, pp. 440–445. [Online]. Available: <https://aclanthology.org/D17-1045>
- [6] A. Ledenev, "Pumba: Chaos testing and network emulation tool for docker," 2026, versión 1.1.7. [Online]. Available: <https://github.com/alexei-led/pumba>